Retail SQL Reconciliation and Evidence-Based Investigation

MiraFlores AI Engineer case study | Jiacheng | Synthetic data, runnable local prototype

1. Three problems and a focused choice

I worked as an SDE Intern at JD for several months, with exposure to tasks related to data engineering. JD's retail and e-commerce business involves connected datasets for orders, products, and reporting. Differences in how these datasets are filtered, joined, or aggregated can lead to inconsistent results.

Recurring issues I encountered included duplicate or missing identifiers, inconsistent formats, and disagreements in reporting conventions. These observations motivate the problems below. The prototype reflects this retail setting through synthetic order-line snapshots, product tables, and order-amount reporting. It does not reconstruct an employer's architecture, incidents, code, or data.

Problem 1: investigating different results from related datasets (selected). In retail data work, engineers and analysts can find that related outputs disagree even though processing jobs completed. Duplicate records, missing join keys, and different processing rules make investigation harder. Unresolved differences can distort aggregates or delay trusted reporting. The recurring data issues are observed; their frequency, business cost, and investigation time have not been measured here.

Problem 2: inconsistent representations across incoming sources. I encountered date and amount format inconsistencies. Engineers must interpret source values before conversion, and analysts depend on the resulting dates and amounts. Unresolved formats can fail processing or assign values to the wrong period. The format differences are observed. The claim that reusable contracts would reduce repeated effort is a hypothesis; source owners must first confirm conventions.

Problem 3: inconsistent refund and cancellation definitions. I also encountered disagreements in reporting conventions. Engineers implementing metrics and people comparing reports are affected. Two technically valid reports can disagree when they count different statuses, dates, or amount definitions. The disagreements are observed; whether shared definitions and acceptance tests would resolve them remains unvalidated.

Why prioritize the first problem? It interests me because investigation connects data engineering with an inspectable outcome: the first observed divergence, affected records, and the SQL responsible for producing each snapshot. A useful initial workflow can be demonstrated without production access. Format checks are a supporting input boundary; resolving competing business definitions requires stakeholder agreement that a standalone tool cannot provide. This prioritizes a tractable experiment, not a claim that the other problems have lower economic impact.

Assumptions to validate. Users have two related pipelines, can supply comparable checkpoints and business requirements, and need help understanding discrepancies. They are willing to review evidence rather than accept an automatic repair. The hypothesis is that an evidence packet reduces investigation effort beyond their existing SQL and checks; no productivity saving is claimed.

2. Approach, alternatives, and scope

Current handling and a better outcome. A credible baseline is manually inspecting SQL, comparing totals, then drilling into records or exporting results to spreadsheets. This is an evaluation baseline, not a verified description of every JD team's workflow. A better outcome combines the common user request, raw and scoped differences, retained records, executed SQL, and a traceable explanation in one reproducible run.

Smallest useful step. Run two explicitly staged SQL pipelines against the same loaded source snapshots. Apply the same configured filters and grouping, compare results and detail records, and examine aligned checkpoints in order. Report the first observed divergence and preserve the intermediate tables. The tool accepts provided stages; it does not automatically decompose arbitrary SQL or discover warehouse lineage.

Conventional software computes the evidence. A validated JSON job declares sources, optional preflight contracts, SQL files, aligned checkpoints, keys, amount columns, dimensions, and query scope. SQLite materializes stages separately and records SQL, dependencies, schemas, and failures. Exact decimal parsing produces integer cents; comparison sums use Python integers. Query filters are parameterized. Duplicate or missing keys remain visible and produce ambiguity rather than a keep-first match.

AI interprets a checked evidence packet. Optional model analysis receives the supplied requirement, actual SQL, computed differences, and declared-join traces. It returns observations, hypotheses, and human verification steps. Local validation checks real evidence IDs, exact paths/values, and an unchanged deterministic status. It cannot prove that the prose follows logically from the citations. The model has no action tools, and suggestions are never executed automatically.

Alternative considered. Adding checks directly to an existing warehouse/orchestrator could reuse production metadata and avoid another tool. That may be the better deployment. I chose a local CLI because it is reproducible with synthetic data and requires no production account. A totals-only comparator would be simpler, but equal totals can hide replaced keys or offsetting changes; the prototype therefore retains detail checks.

Connection to the broader opportunity. If the local workflow proves useful, a warehouse adapter could collect the same snapshots and evidence automatically. Before that, I would learn how users align checkpoints, who owns metric definitions, how much evidence is sufficient, and whether their existing tools already solve this problem. General SQL parsing, catalog integration, a UI, automatic fixes, and scale optimization are deferred.

3. Working prototype and demonstration

Run without an account: `python3 demo_reconciliation.py`. This executes real SQL and comparison logic, checks two supplied corrections, demonstrates a query masking a raw difference, and identifies a deliberately invalid SQL step. New run directories preserve prior outputs. See README.md and docs/DEMO.md for setup, individual commands, expected exits, and a 15-minute walkthrough.

Example	Computed evidence and meaningful output	Result
Filter mismatch	B retains cancelled I003; A totals 30000 cents, B 177900; Beijing differs by +147900	DIFFERENT; first divergence at filtered
History join fanout	B uses all product versions; I001 has two copies; totals rise from 30000 to 40000 cents	AMBIGUOUS; first divergence at enriched
Paid-only user request	Query excludes cancelled I003; raw datasets still differ by 147900 cents	Scoped MATCH; raw divergence retained
Invalid SQL	B/filter selects a nonexistent column; A's completed tables remain inspectable	ERROR; identifies B/filter

Representative investigation. The paid-only requirement and the two filter SQL files explain why I003 needs attention. A manually supplied corrected B filter produces MATCH and zero amount difference. This is an acceptance fixture, not an automatically generated or independently validated repair. The signed difference consistently means B minus A.

Difficult case. Product history contains a legacy and a current P01 row. Both pipelines prepare current products, but B's enrichment accidentally reads the original history table. Diagnostics preserve both right matches and both I001 outputs, with business keys, SQLite row IDs, and values. Joining the prepared current-products table restores MATCH while retaining all four historical source rows. The demo checks unchanged source-file hashes. Repeated keys are not overwritten or arbitrarily deduplicated.

Artifacts. Each run retains run.sqlite, execution.json, reconciliation.json, difference CSVs, report.md, and evidence.json. Evidence IDs resolve to exact saved JSON values. Optional ai/ files retain the request, response, local validation outcome, and interpretation. A model failure leaves the computed report available.

Verification. The final suite has 71 offline tests, including monetary precision/type failures, equal aggregates with differing details, query masking, ambiguous keys, join uncertainty, retained failures, and rejected AI citations. Packaging tests check file exclusion, unsafe paths, and required AI resources in an extracted package. Runtime does not read the manual expected.json answer keys. All samples are synthetic; no warehouse integration or production reliability is claimed.

4. Evaluation, next learning, and disclosure

Real model evaluation. Three final live cases used gpt-5-mini: filter mismatch, join fanout, and matching query results over different raw datasets. They passed 10, 12, and 11 exact anchor checks respectively. Seven live analysis calls were made in total: one initial accepted response, three rejected during interface refinement, then the three final accepted responses. Captured requests/responses and review findings are included; the application never uses them as runtime fallback answers.

What the evaluation exposed. An early response cited valid evidence but described duplicate business-key projections as identical complete rows. Other attempts used the wrong pointer convention and were rejected. The prompt and interface were refined to make /data paths explicit, read actual dependencies, preserve duplicates, and avoid unsupported intent claims. Remaining issues include computed facts appearing in the hypotheses section and incomplete support for contextual wording. Structured output and exact citations are useful controls, not a guarantee of a correct diagnosis.

What I would investigate next. Observe engineers investigating recent non-sensitive discrepancies. Compare their current method, the deterministic report alone, and the optional AI interpretation on held-out cases. Measure review time, actionable findings, unsupported claims, false alarms, and maintenance effort. Test changed business requirements, multiple simultaneous defects, ambiguous grain, and instruction-like text in task data. A broader opportunity depends on useful evidence collection and clear ownership, not merely adding a model.

When I would stop. Deprioritize the tool if existing warehouse checks already deliver this evidence, discrepancies are rare, checkpoints cannot be aligned economically, or reviewers spend more time correcting the explanation than investigating manually. Defer the AI feature if it adds no value beyond report.md or produces unacceptable unsupported claims.

Implemented, mocked, and deferred. SQL execution, reconciliation, join observations, evidence reports, optional real-provider analysis, and packaging run. Inputs and supplied correction variants are synthetic. Offline AI tests inject explicitly labeled stubs; there is no runtime mock mode. General lineage, warehouse access, automatic fixes, and large-data performance remain deferred.

AI and reuse disclosure. Codex assisted with ideation, code, fixtures, tests, documentation, and review. Verification combined manually specified arithmetic and expected records, automated execution, source-hash checks, exact evidence-pointer checks, and review of real responses. These AI-assisted checks are not independent proof of customer value. No prior project code was reused. Source preflight adapts validation code developed earlier within this exercise; the standalone CSV workflow and its fixtures were removed. SQLite and the optional OpenAI SDK/PDF library are third-party dependencies, not employer work.

Time spent. The recorded development window spans approximately 201.0 elapsed minutes, including gaps and potentially overlapping review. Timer-covered implementation totals 46.4 minutes; the applicant separately reported approximately 30 minutes of reading, review, and understanding, giving a disclosed subtotal of approximately 76.4 minutes. Earlier discussion and final publication were not fully timed, so that subtotal omits some work. See docs/TIME_DISCLOSURE.md and WORK_LOG.md. Git history records milestones rather than continuous effort.